DEPARTMENT OF COMPUTER SCIENCE AND ENGINEEERING (CYBER SECURITY)

CS23621 MOBILE APPLICATION DEVELOPMENT LABORATORY

RAJALAKSHMI ENGINEERING COLLEGE

Thandalam, Chennai-602015

RAJALAKSHMI ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

CS23621 Mobile Application Development Laboratory

LAB MANUAL

2023 REGULATION

SIXTH SEMESTER

III YEAR

COLLEGE VISION

- To be an institution of excellence in Engineering, Technology and Management, Education & Research.
- To provide competent and ethical professionals with a concern for society.

COLLEGE MISSION

- To impart quality technical education imbibed with proficiency and humane values.
- To provide right ambience and opportunities for the students to develop into creative, talented and globally competent professionals.
- To promote research and development in technology and management for the benefit of the society.

DEPARTMENT VISION

To promote highly ethical and innovative cybersecurity professionals through excellence in teaching, training, and research.

DEPARTMENT MISSION

- To produce globally competent cybersecurity experts, motivated to learn emerging technologies and be innovative in solving real-world security challenges.
- To foster research activities among students and faculty that enhance societal security and resilience.
- To impart moral and ethical values in their profession, ensuring a strong commitment to cybersecurity ethics and responsible practices.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEO 1: To equip students with essential background in computer science, basic electronics and applied mathematics.

PEO 2: To prepare students with fundamental knowledge in programming languages and tools and enable them to develop applications.

PEO 3: To encourage the research abilities and innovative project development in the field of networking, security, data mining, web technology, mobile communication and also emerging technologies for the cause of social benefit.

PEO 4: To develop professionally ethical individuals enhanced with analytical skills, communication skills and organizing ability to meet industry requirements.

PROGRAMME OUTCOMES (POs)

PO1: Engineering knowledge: Apply the knowledge of Mathematics, Science, Engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leaderin a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

A graduate of the Computer Science and Engineering Program will demonstrate

PSO 1: Foundation Skills: Ability to understand, analyze, and develop computer programs in the areas related to algorithms, system software, web design, machine learning, data analytics, networking, and cybersecurity for efficient design of computer-based systems of varying complexity. Familiarity and practical competence with a broad range of programming languages, open-source platforms, and cybersecurity tools and practices.

PSO 2: Problem-Solving Skills: Ability to apply mathematical methodologies to solve computational tasks, model real-world problems using appropriate data structures and suitable algorithms, and address cybersecurity challenges. Understanding of standard practices and strategies in software project development, incorporating secure coding practices and using open-ended programming environments to deliver quality and secure products.

PSO 3: Successful Progression: Ability to apply knowledge in various domains to identify research gaps and provide solutions to new ideas, with a particular emphasis on cybersecurity challenges. Inculcating a passion for higher studies, creating innovative career paths to be an entrepreneur, and evolving as an ethically and socially responsible computer science professional with strong cybersecurity expertise.

Subject Code	Subject Name (Lab Oriented Course)	Category	L	T	P	C
CS23621	Mobile Application Development Laboratory	PC	0	0	4	2

Objectives:
• Get started developing in Kotlin, and learn the basics of the Kotlin programming language: data types, operators, variables, control structures, and nullable versus non-nullable variables.
• Get an introduction to Android development and UI basics.
• Learn Android app architecture using Kotlin.
• Learn best practices, guidelines, and tools for effective Android app design.
• To understand the capabilities and limitations of mobile devices

List of Experiments		
1.	Kotlin Basics	
2.	Functions	
3.	Classes and Objects	
4.	Build First Android App	
5.	Layouts	
6.	App Navigation	
7.	Activity and Fragment Lifecycles	
8.	App Architecture (UI Layer)	
9.	App Architecture (Persistence)	
10.	Advanced RecyclerView Use Cases	
11.	Connect to the Internet	
12.	Repository Pattern and WorkManager	
13.	App UI Design	
14.	Mini Project	
	Contact Hours:	**30**
	Total Contact Hours :	**60**

Course Outcomes: On completion of the course, students will be able to
• Learn the basics of Kotlin, including data types, operators, variables, control structures, and handling nullable and non-nullable variables.
• Develop Android apps and the basics of user interface design.
• Become familiar with the architecture of Android apps and how to structure them using Kotlin.
• Learn the best practices, guidelines, and essential tools needed for designing effective Android apps.
• Develop an understanding of what mobile devices can and cannot do, which is crucial for mobile app development.

Ex. No : 1

Kotlin Basics

Aim

To understand the basics of Kotlin programming including variables, data types, and simple input/output operations.

Procedure

1. Open Android Studio and create a new Kotlin project:
    - Launch Android Studio.
    - Click New Project → Empty Activity → Kotlin as the programming language.
    - Give your project a name (e.g., KotlinBasics).
    - This sets up the environment where you can write and run Kotlin programs.
2. Write a Kotlin program to demonstrate variables, constants, and data types:
    - In Kotlin, variables are used to store data.
    - Variables can be declared using var (mutable) or val (immutable/constant).
    - Kotlin is statically typed, which means the type of data is defined at compile time.
3. Run the program to check output:
    - Click the Run button in Android Studio.
    - Observe the output in the Run Console.
    - The program prints the values stored in variables to the console.

STEP 1: Open Android Studio

1. Open Android Studio (Otter / Otter 2)
2. Click New Project

STEP 2: Create New Project

1. Select Empty Activity
2. Click Next

Field	Value
Name	**KotlinBasics**
Language	**Kotlin**
Minimum SDK	**API 21 or above**
Build Configuration	**View-based (NOT Compose)**

STEP 3: Open Layout File (TextView UI)

app → res → layout → activity_main.xml

STEP 4: Add TextView

Create Layout XML (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp">
    <TextView
        android:id="@+id/txtOutput"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Output will appear here"
        android:textSize="20sp" />
</LinearLayout>
```

STEP 5: Open Kotlin File

app → java → com.example.kotlinbasics → MainActivity.kt

MainActivity.kt

```
package com.example.kotlinbasics

import android.os.Bundle

import android.widget.TextView

import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        // Reference to TextView

        val output = findViewById<TextView>(R.id.txtOutput)

        // Kotlin Basics

        val name = "Android"

        val a = 10

        val b = 20

        val sum = a + b

        // Display output

        output.text = "Welcome $name\nSum = $sum"

    }

}
```

STEP 7: Create Emulator

- Click Device Manager
- Click Create Device
- Choose Pixel / Medium Phone
- Select Android version
- Click Finish
- Start emulator ■

STEP 8: Run the App

1. Select emulator
2. Click Run ■
3. App opens on emulator

Output

Viva -Questions

1. What is Kotlin?

Kotlin is a modern, statically typed programming language officially supported for Android app development**.**

2. Why Kotlin is preferred for Android?

It is concise, safe, and fully interoperable with Java.

3. What is an Activity?

An Activity represents a single screen in an Android application.

4. Why main() is not used in Android?

Android apps start execution from lifecycle methods like onCreate().

5. What is onCreate()?

It is the first lifecycle method called when an Activity is created.

6. Difference between val and var?

val is immutable (cannot be changed), var is mutable.

7. What is setContentView()?

It connects the Kotlin file with the XML layout.

8. What is TextView?

TextView is a UI component used to display text on the screen.

9. Why output is not shown using println()?

Android apps do not use console output.

10. Where is UI designed in Android?

In XML layout files inside the res/layout folder.

Ex. No :02

Functions

Aim

To understand and implement **functions in Kotlin**, including **defining functions with parameters, return types, and calling them**, and display output in an Android app.

Explanation

- A function is a block of code designed to perform a specific task.
- In Kotlin, functions are defined using the fun keyword.
- Functions can:
    1. Take parameters (inputs).
    2. Return a value (Int, String, etc.) or Unit (no return).
    3. Be called multiple times to reuse code.
- In Android apps, all code must be inside a class (usually MainActivity) and runtime code goes inside onCreate().
- Output is displayed using a TextView, instead of println() like in console programs.

Procedure

1. Open Android Studio .
2. Open the project and go to res/layout/activity_main.xml.
3. Add a TextView to display output.
4. Open MainActivity.kt and:
    - Define functions inside the class.
    - Call the functions inside onCreate().
    - Set the output to the TextView.
5. Run the app on an emulator or physical device.
6. Observe the output displayed on the screen.

Notes :

1. All functions are inside the MainActivity class.
2. TextView is used to display output instead of println().
3. Functions demonstrate:
    - Parameter passing (add(a, b), greet(name))
    - Returning values (Int, String)
    - Function without parameters (welcomeMessage())

Code

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp">
    <TextView
        android:id="@+id/txtOutput"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Kotlin Output"
        android:textSize="20sp" />
</LinearLayout>
```

MainActivity.kt

```
package com.example.kotlinbasics

import android.os.Bundle

import android.widget.TextView

import androidx.appcompat.app.AppCompatActivity


class MainActivity : AppCompatActivity() {


    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)


        val txt = findViewById<TextView>(R.id.txtOutput)


        val a = 10

        val b = 20

        val sum = a + b


        txt.text = "Welcome to Kotlin\nSum = $sum"

    }

}
```

Output

Viva -Questions

1. What is a function?

A function is a reusable block of code that performs a specific task.

2. Why functions are used?

To reduce code repetition and improve readability.

3. Syntax of a Kotlin function?

fun functionName(): ReturnType

4. What is return type?

It specifies the type of value returned by a function.

5. What is a parameter?

A parameter is a value passed to a function.

6. Can a function return multiple values?

No, but Kotlin supports data classes and pairs.

7. What is setOnClickListener()?

It handles button click events.

8. Difference between function and method?

A function is general; a method belongs to a class**.**

9. When is a function executed?

When it is called.

10. What happens if return type is not mentioned?

The return type becomes Unit.

Ex. No. :03

Classes and Objects

Aim

To understand the concept of **classes and objects in Kotlin**, including **creating classes, defining properties and methods**, and using **objects to access class members** in an Android app.

Explanation

- A class is a blueprint for creating objects.
- An object is an instance of a class.
- Classes can have:
    1. Properties (variables inside class)
    2. Methods/Functions (actions the class can perform)

In Kotlin:

```
class Person(val name: String, var age: Int) {
   fun greet() = "Hello, $name!"
}
```

In Android, we create objects inside MainActivity and use a TextView to display output.

Procedure :

1. Open activity_main.xml and ensure there is a TextView for output:
2. Open MainActivity.kt.
3. Define a class inside the Kotlin file or in a separate file.
4. Create an object of the class inside onCreate().
5. Call class methods and display output in the TextView.
6. Run the app to see the results

Notes

1. Person class has:
    - Property name (immutable)
    - Property age (mutable)
    - Method greet() returns a greeting
    - Method birthday() updates age
2. Object person is used to access class properties and methods.
3. Output is displayed in TextView, suitable for Android apps.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <!-- App Toolbar -->
    <com.google.android.material.appbar.MaterialToolbar
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:title="Kotlin Classes"
        android:titleTextColor="@android:color/white"
        android:background="?attr/colorPrimary" />
    <!-- Main Content -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:gravity="center"
```

```
        android:padding="24dp"
        android:background="#FAFAFA">

        <Button
            android:id="@+id/btnShow"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Show Student Details" />

        <TextView
            android:id="@+id/txtOutput"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Output will appear here"
            android:textSize="18sp"
            android:padding="16dp"
            android:layout_marginTop="16dp"
            android:background="@android:color/white"
            android:elevation="6dp" />
    </LinearLayout>
</LinearLayout>
```

Create a Kotlin Class

- Right-click package name
- Select New → Kotlin Class/File
- Name it: Student
- Choose Class

Write Student.kt

```
package com.example.kotlinbasics


class Student(
    val name: String,
    val rollNo: Int
) {
    fun displayDetails(): String {
        return "Name: $name\nRoll No: $rollNo"
    }
}
```

Update strings.xml

res → values → strings.xml

```
<string name="student_output">Student Details\n%1$s</string>
```

Code

MainActivity.kt

```
package com.example.kotlinbasics
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btn = findViewById<Button>(R.id.btnShow)
        val txt = findViewById<TextView>(R.id.txtOutput)

        btn.setOnClickListener {
            // Creating object of Student class
            val student = Student("Arun", 101)

            txt.text = getString(
                R.string.student_output,
                student.displayDetails()
            )
        }
    }
}
```

STEP 7: Run the App

1. Start emulator
2. Click Run ▶
3. Tap Show Student Details

Output :


Show Student Details
Output will appear here
Show Student Details
Student Details
Name: Arun
Roll No: 101

Viva -Questions

1. **What is a class?**

A class is a blueprint for creating objects.

2. **What is an object?**

An object is an instance of a class.

3. **How to create an object in Kotlin?**

val obj = ClassName()

4. **What is a constructor?**

A constructor initializes the properties of a class.

5. **What is a primary constructor?**

It is defined in the class header.

6. **What are properties?**

Variables declared inside a class.

7. **What are member functions?**

Functions defined inside a class.

8. **Can a class have multiple objects?**

Yes, multiple objects can be created from a single class.

9. **What is encapsulation?**

Wrapping data and methods together inside a class.

10. **Difference between class and object?**

Class is a template; object is the real instance.

Ex. No : 04

Build Your First Android App

Aim

To create a simple Android app using Kotlin, understand Activity, layout, and display output on screen.

Prerequisites:

1. Android Studio installed.
2. Kotlin selected as the programming language.
3. Basic understanding of Activities and Layouts.

Explanation

- An Android app is composed of:
    1. Activities – Screens that users interact with (MainActivity)
    2. Layout XML – Defines the UI components (TextView, Button, etc.)
    3. Manifest – Declares the app components and permissions (AndroidManifest.xml)
- MainActivity is the entry point of the app.
- setContentView(R.layout.activity_main) connects the Activity with its layout.
- Output is displayed on TextView in the UI instead of console output.

Procedure

1. Open Android Studio Otter 2.
2. Create a new project → Empty Activity → Kotlin → Finish.
3. Open res/layout/activity_main.xml and add a TextView:
4. Open MainActivity.kt.
5. Inside onCreate(), find the TextView using findViewById and set its text.
6. Run the app on an **emulator or physical device**.
7. Observe the output on the app screen.

Step 1: Create a New Project

1. Open **Android Studio** → Click **"New Project"**.
2. Select **Empty Activity** → Click **Next**.
3. Give your project a name, e.g., MyFirstApp.
4. Set **Language = Kotlin**.
5. Minimum SDK → Choose **API 21: Android 5.0 (Lollipop)** or higher.
6. Click **Finish**.

Step 2: Design the Layout

1. **Open res/layout/activity_main.xml.**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    android:gravity="center">
    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, Android!"
        android:textSize="24sp"
        android:textColor="#000000"
        android:padding="16dp"/>
```

```
        <Button
            android:id="@+id/btnClick"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Click Me"/>
    </LinearLayout>
```

Step 3: Add Logic in Kotlin

Code

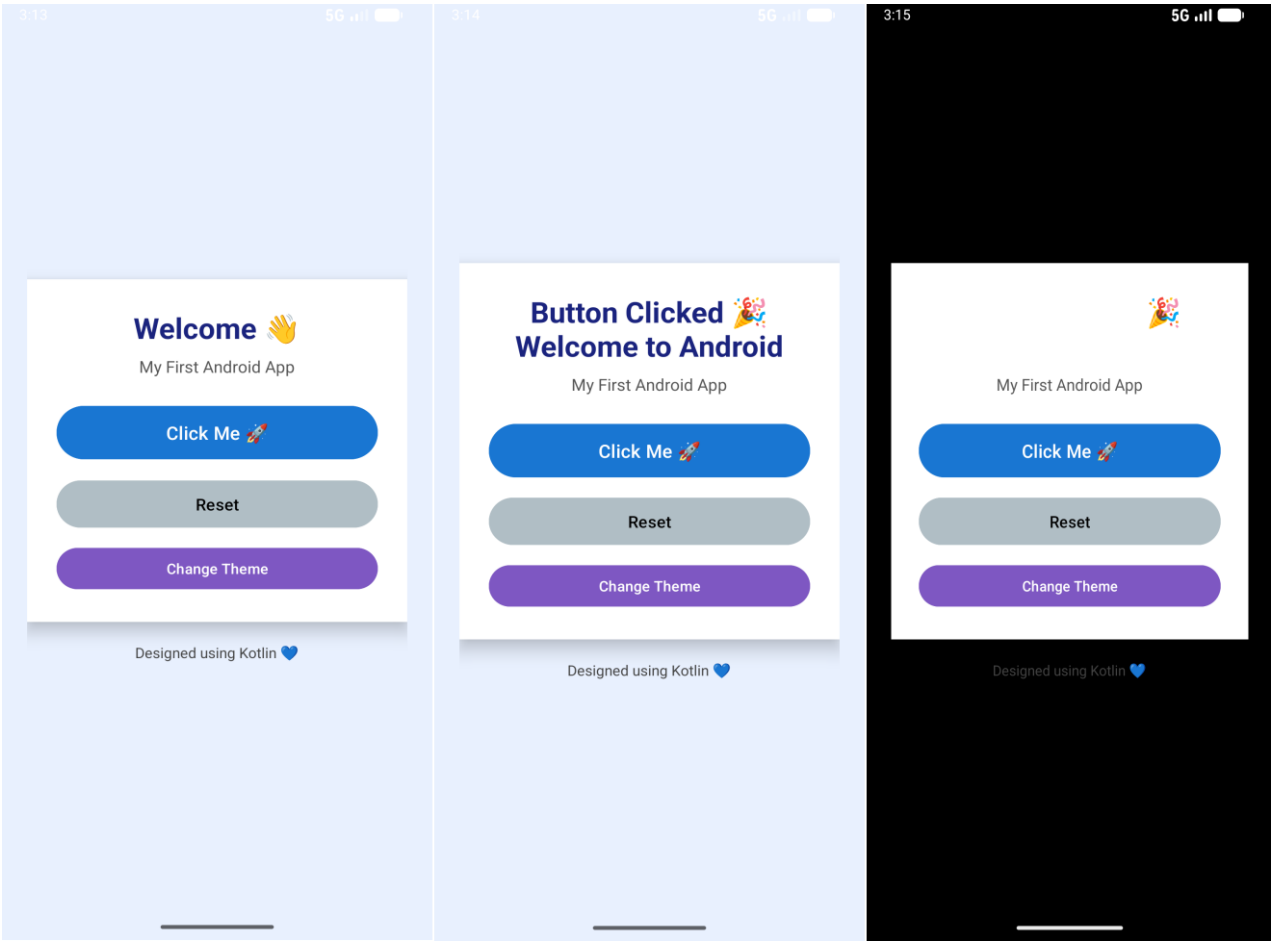
MainActivity.kt

```
package com.example.myfirstapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnClick = findViewById<Button>(R.id.btnClick)

        btnClick.setOnClickListener  {
            tvMessage.text = "Button Clicked! Welcome to Android."
        }
    }
}
```

Step 4: Run the App

1. Connect an Android device or use an emulator.
2. Click Run (▶) in Android Studio.
3. You should see:
    - Initial text: Hello, Android!
    - On button click → Text changes to Button Clicked! Welcome to Android.

Output



Viva – Questions

1. What is an Activity in Android?

An Activity represents a single screen in an Android app, similar to a window in desktop applications.

It handles UI and user interactions.

2. What is the purpose of setContentView() in onCreate()?

It sets the layout XML file to be displayed by the Activity.

Example: setContentView(R.layout.activity_main) links activity_main.xml to the Activity**.**

3. How do you link a UI element from XML to Kotlin code?

Using findViewById().

Example: val btn = findViewById<Button>(R.id.btnClick)

4. What is an OnClickListener?

It is used to detect click events on a Button or other clickable UI elements.

5. How do you change the text of a TextView programmatically?

Using the .text property.

Example: tvMessage.text = "Button Clicked!"

6. What is the difference between wrap_content and match_parent?

- wrap_content: UI element size adjusts to fit its content.
- match_parent: UI element fills the parent layout's available space.

7. What is the role of LinearLayout?

It arranges UI elements vertically or horizontally depending on android:orientation.

8. Why do we use Kotlin for Android app development?

Kotlin is concise, safe (null-safe), interoperable with Java, and officially supported by Google for Android development.

9. Can you explain the lifecycle method used in this experiment?

We used onCreate(), which is called when the Activity is first created. It is used to initialize UI elements and set the layout.

10. How would you handle multiple button clicks?

Assign setOnClickListener to each button or implement a single listener and use when(view.id) to handle multiple buttons.

Ex.No : 5

Layouts

Aim

To understand different types of layouts in Android, including LinearLayout, RelativeLayout, and ConstraintLayout, and design user interfaces**.**

Android Layouts define how UI elements are arranged on the screen.

Common layouts:

1. LinearLayout – Arranges elements vertically or horizontally.
2. RelativeLayout – Elements positioned relative to each other or parent.
3. ConstraintLayout – Modern, flexible layout with constraints.
4. FrameLayout – Stack elements on top of each other.
5. TableLayout – Arrange elements in rows and columns.

Procedure:

1. Open activity_main.xml.
2. Design UI using LinearLayout with multiple TextViews, Buttons, or ImageViews.
3. Experiment with orientation (vertical/horizontal) and padding/margin.
4. Try RelativeLayout to position elements relative to each other.
5. Try ConstraintLayout to create more flexible layouts using constraints.
6. Run the app on an emulator/device and observe the UI.

Step 1: Create a New Project

- Open Android Studio → New Project → Empty Activity
- Name: LayoutsEx5
- Language: Kotlin
- Minimum API: 21+
- Click Finish

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:padding="16dp">
    <TextView
        android:id="@+id/tvWelcome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome to Layouts Ex-5"
        android:textSize="24sp"
        android:textStyle="bold" />
    <Button
        android:id="@+id/btnNext"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Go to Second Activity"
        android:layout_marginTop="20dp" />

</LinearLayout>
```

Step 3: Create Second Activity

- Right-click app → New → Activity → Empty Activity
- Name: SecondActivity
- Ensure Generate Layout File is checked → Click Finish

Step 4: Design SecondActivity Layout

File: res/layout/activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvSecond"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is the Second Activity"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

Step 5: Add Button Functionality in MainActivity

MainActivity.kt

```
package com.example.layoutsex5

import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Find the button by ID
        val btnNext = findViewById<Button>(R.id.btnNext)

        // Set OnClickListener
        btnNext.setOnClickListener {
            // Navigate to SecondActivity
            val intent = Intent(this, SecondActivity::class.java)
            startActivity(intent)
        }
    }
}
```

Step 6: Ensure SecondActivity Code

File: SecondActivity.kt

```
package com.example.layoutsex5


import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle


class SecondActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_second)
    }
}
```

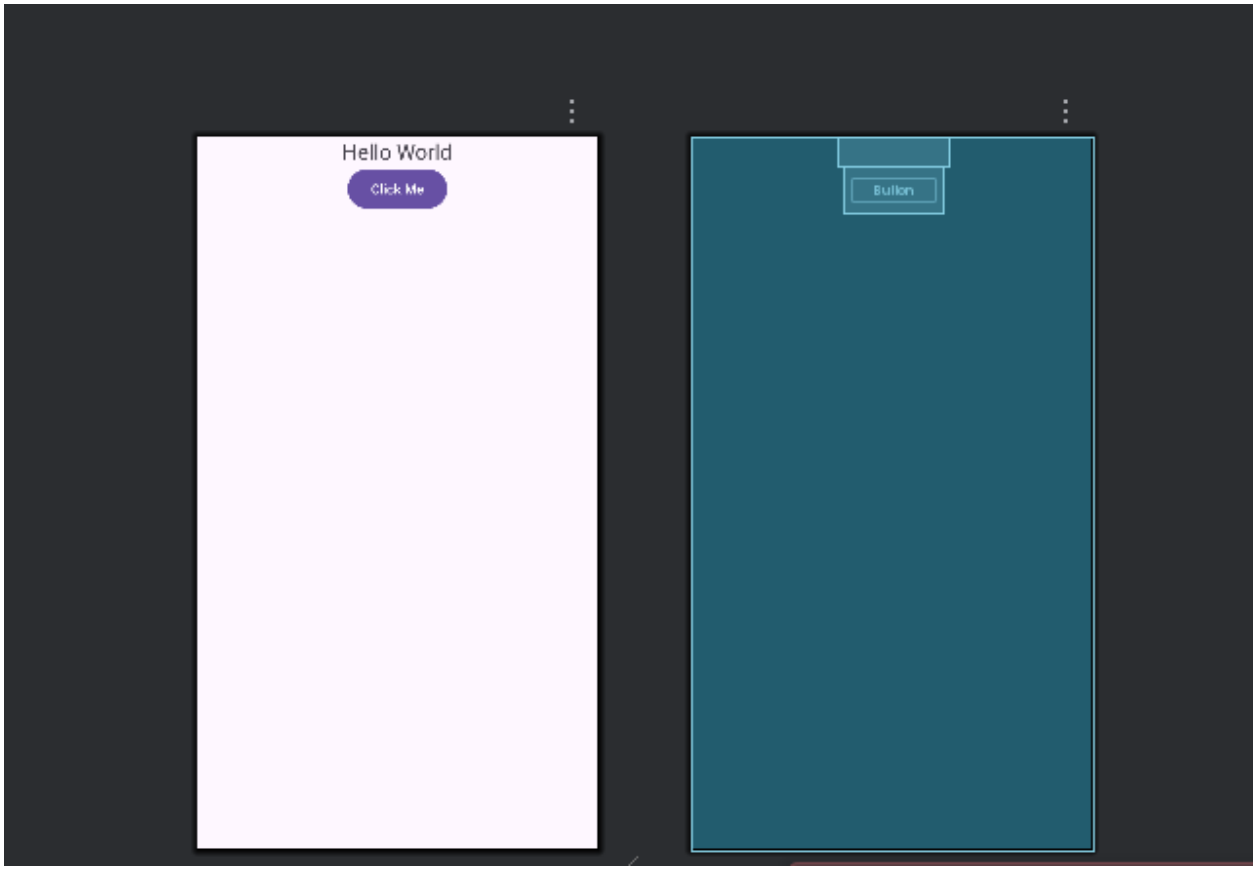
Step 7: Update AndroidManifest

```
<application
    android:allowBackup="true"
    android:label="@string/app_name"
    android:theme="@style/Theme.LayoutsEx5">
    <activity android:name=".SecondActivity"/>
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
    </activity>
</application>
```

Step 9: Run the App

- Connect a device/emulator
- Click Run → Run 'app'
- Check the output:
    1. MainActivity shows TextView + Button
    2. Click button → opens SecondActivity

Output



Viva - Questions

1. What is a layout in Android?

A layout defines the structure and arrangement of UI components on the screen.

2. Where are layouts stored?

Layouts are stored in the res/layout folder as XML files.

3. Why XML is used for layouts?

XML separates UI design from program logic and is easy to maintain**.**

4. What is activity_main.xml?

It is the default layout file associated with an Activity.

5. What is LinearLayout?

LinearLayout arranges UI components in a single row or column.

6. What is the use of orientation attribute?

It specifies whether the layout is horizontal or vertical.

7. What is RelativeLayout?

RelativeLayout positions UI components relative to each other or parent.

8. What is ConstraintLayout?

ConstraintLayout positions UI elements using constraints, making flexible and responsive UI.

9. Which layout is preferred in modern Android?

ConstraintLayout is preferred for complex and responsive designs.

10. What is layout_width and layout_height?

They define the size of a UI component.

Ex. No.: 06

App Navigation

Aim

To understand navigation between multiple screens in an Android application using **Intents** and **multiple Activities**.

Explanation :

- In Android, an Activity represents a single screen with a user interface.
- Most Android applications consist of multiple screens, such as:
    - Login screen
    - Home screen
    - Details screen

Why Navigation is Needed

- Navigation allows users to move from one screen to another.
- Android uses an Intent to perform this task**.**

Intent – Core Concept

- An Intent is a messaging object used to:
    - Start another Activity
    - Pass data between Activities
- There are two types of intents:
    - Explicit Intent – Used to move between activities within the same app
    - Implicit Intent – Used to perform system actions (call, open browser, etc.)

How Navigation Works

1. User clicks a Button in the first Activity.
2. OnClickListener listens for the click event.
3. An Intent object is created with:
    - Current Activity
    - Target Activity
4. startActivity() launches the new screen.

Procedure

1. Open Android Studio and create a project using Empty Activity.
2. Design the first screen with a Button.
3. Create a Second Activity.
4. Add a TextView to the second Activity layout.
5. Write Kotlin code to handle Button click.
6. Use an Intent to navigate from the first Activity to the second Activity.
7. Run the app and test navigation.

Step 1: Create a New Project

1. Open Android Studio → New Project → Empty Activity.
2. Name the project: AppNavigationDemo.
3. Language: Kotlin. Minimum SDK: API 21 or higher.
4. Click Finish.

Step 2: Create a Second Activity

1. Right-click on app → java → <your package> → New → Activity → Empty Activity.
2. Name it SecondActivity.
3. Click Finish.

Step 3: Design Layouts

1. MainActivity Layout (activity_main.xml)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvMain"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome to Main Activity"
        android:textSize="24sp"
        android:padding="16dp"/>

    <Button
        android:id="@+id/btnNext"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Go to Second Activity"/>
</LinearLayout>
```

2. SecondActivity Layout (activity_second.xml)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvSecond"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome to Second Activity"
        android:textSize="24sp"
        android:padding="16dp"/>
</LinearLayout>
```

Step 4: Add Navigation Logic in Kotlin

1. MainActivity.kt

```
package com.example.appnavigationdemo
import android.content.Intent
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
```

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnNext = findViewById<Button>(R.id.btnNext)

        btnNext.setOnClickListener {
            // Intent to navigate to SecondActivity
            val intent = Intent(this, SecondActivity::class.java)
            startActivity(intent)
        }
    }
}
```

2. SecondActivity.kt

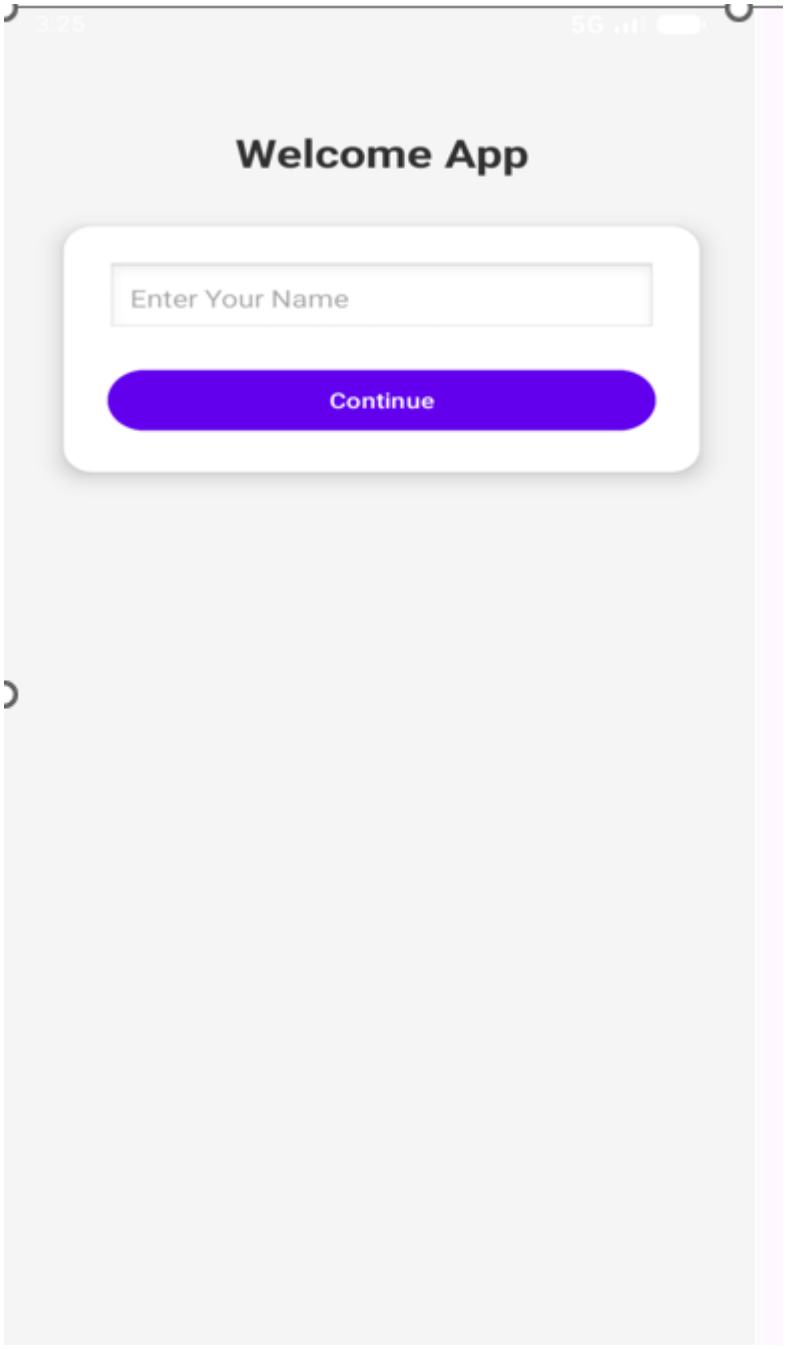
```
package com.example.appnavigationdemo

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class SecondActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_second)
    }
}
```

Step 5: Run the App

1. Connect an Android device or use an emulator.
2. Click Run (▶) in Android Studio.
3. You should see MainActivity.
4. Click “Go to Second Activity” → SecondActivity appears.

Output

- When the application starts, Main Activity is displayed.
- Clicking the “Go to Second Screen” button opens Second Activity.
- The second screen displays the message**:**



Viva - Question

1. What is App Navigation?

App Navigation is the process of moving between different screens (Activities or Fragments) in an Android app.

2. Why is navigation important in Android?

It improves user experience by organizing app flow and screen transitions.

3. What are the main components used for navigation?

- Activity
- Intent
- Fragment
- Navigation Component

4. What is an Intent?

An Intent is a messaging object used to request an action from another component.

5. Types of Intents?

- Explicit Intent – specifies target Activity
- Implicit Intent – does not specify target component

6. Which Intent is used in App Navigation experiment?

Explicit Intent is used.

7. Syntax to start another Activity?

val intent = Intent(this, SecondActivity::class.java)

startActivity(intent)

8. What is an Activity?

An Activity represents a single screen with UI.

9. How do you navigate from one Activity to another?

By using startActivity() with an Intent.

10. Where are Activities declared?

In the AndroidManifest.xml file.

Ex.No : 7

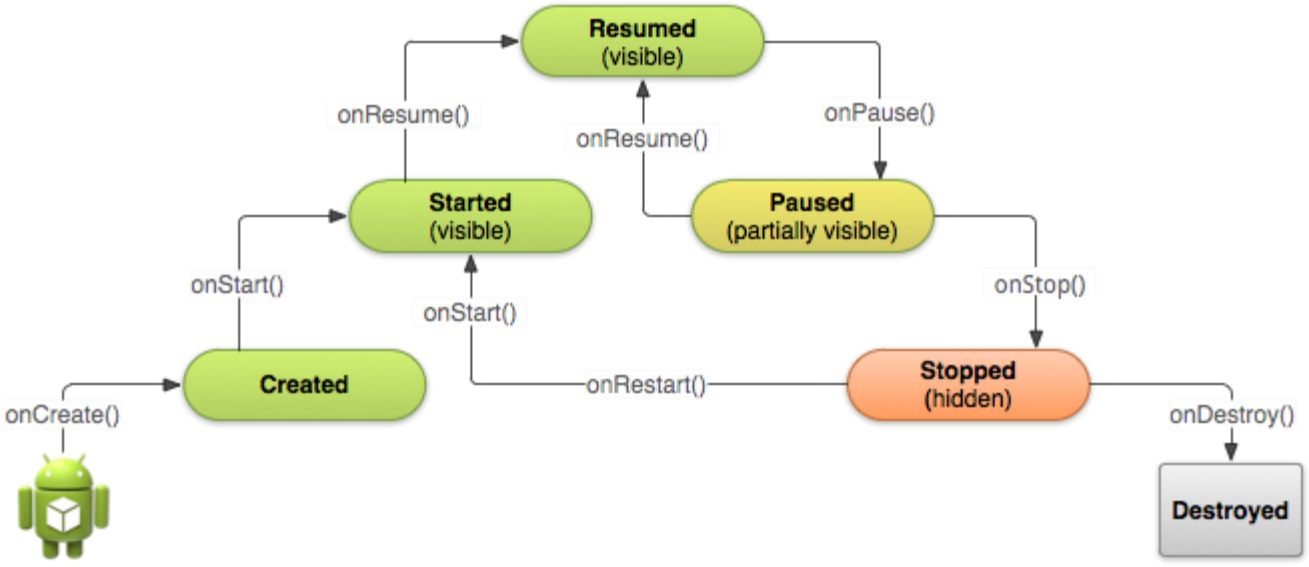
Activity and Fragment Lifecycles

Aim

To understand the **lifecycle of an Activity and Fragment** in Android and observe how different lifecycle methods are invoked during various states of the application.

What is Lifecycle?

- The lifecycle refers to the different states an Activity or Fragment goes through from creation to destruction.
- Android automatically calls lifecycle methods based on user actions, such as:
    - Opening the app
    - Switching apps
    - Rotating the device
    - Closing the app



Why Lifecycle Management is Important

- To manage resources efficiently
- To save and restore user data
- To avoid memory leaks
- To ensure smooth app behavior during background/foreground transitions

Activity Lifecycle Methods

Method	Purpose
onCreate()	**Activity is created**
onStart()	**Activity becomes visible**
onResume()	**Activity starts interacting with user**
onPause()	**Activity partially hidden**
onStop()	**Activity no longer visible**
onDestroy()	**Activity is destroyed**

Fragment Lifecycle (Brief)

- Fragment lifecycle is similar to Activity but depends on the host Activity.
- Important methods include:
    - onCreateView()
    - onStart()
    - onResume()
    - onPause()
    - onStop()
    - onDestroyView()

How Lifecycle is Observed

- Lifecycle events are observed using:
    - Logcat messages
    - TextView updates

Procedure

1. Create an Android project using Empty Activity.
2. Open MainActivity.kt.
3. Override Activity lifecycle methods.
4. Add Log statements in each lifecycle method.
5. Run the app.
6. Perform actions:
    - Open app
    - Press Home button
    - Reopen app
    - Rotate screen
    - Close app
7. Observe lifecycle callbacks in Logcat.

STEP 1: Create New Project

1. Open Android Studio
2. Click New Project
3. Select Empty Views Activity
4. Click Next
5. Enter:
    - Name: LifecycleDemo
    - Language: Kotlin
    - Minimum SDK: API 21
6. Click Finish
7. Wait until Build Successful

STEP 2: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"/>

    <TextView
        android:id="@+id/tvLifecycle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Lifecycle Output:"
        android:textSize="16sp"
        android:padding="10dp"/>
</LinearLayout>
```

STEP 3: Edit MainActivity.kt

```
package com.example.lifecycledemo


import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity


class MainActivity : AppCompatActivity() {


    lateinit var tvLifecycle: TextView


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)


        tvLifecycle = findViewById(R.id.tvLifecycle)
        tvLifecycle.append("\nActivity onCreate")


        supportFragmentManager.beginTransaction()
            .replace(R.id.fragment_container,  DemoFragment())
            .commit()
    }


    override fun onStart() {
        super.onStart()
        tvLifecycle.append("\nActivity  onStart")
    }
```

```
    override fun onResume() {
        super.onResume()
        tvLifecycle.append("\nActivity  onResume")
    }

    override fun onPause() {
        super.onPause()
        tvLifecycle.append("\nActivity  onPause")
    }

    override fun onStop() {
        super.onStop()
        tvLifecycle.append("\nActivity  onStop")
    }

    override fun onDestroy() {
        super.onDestroy()
        tvLifecycle.append("\nActivity  onDestroy")
    }
}
```

STEP 4: Create Fragment Class

1. Right-click package name
2. Select New → Kotlin Class/File
3. Name: DemoFragment
4. Type: Class
5. Click OK

STEP 5: Write DemoFragment.kt Code

```
package com.example.lifecycledemo


import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment


class DemoFragment : Fragment() {


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        (activity as MainActivity).tvLifecycle.append("\nFragment onCreate")
    }


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        (activity as MainActivity).tvLifecycle.append("\nFragment onCreateView")
        return inflater.inflate(R.layout.fragment_demo, container, false)
    }
    override fun onResume() {
        super.onResume()
        (activity as MainActivity).tvLifecycle.append("\nFragment onResume")
    }
```

```
    override fun onPause() {
        super.onPause()
        (activity as MainActivity).tvLifecycle.append("\nFragment onPause")
    }

    override fun onDestroyView() {
        super.onDestroyView()
        (activity as MainActivity).tvLifecycle.append("\nFragment onDestroyView")
    }
    override fun onDetach() {
        super.onDetach()
        (activity as MainActivity).tvLifecycle.append("\nFragment onDetach")
    }
}
```

STEP 6: Create Fragment Layout

res → layout → New → Layout Resource File

- File name: fragment_demo
- Root element: TextView

```
<?xml version="1.0" encoding="utf-8"?>
<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:text="Fragment Lifecycle Demo"
    android:textSize="22sp"
    android:gravity="center"/>
```

STEP 7: Run Using Device Manager

- Click Device Manager
- Start an Emulator
- Click Run ▶
- App opens on device

Output

Viva -Questions

1. **What is an Activity lifecycle?**

It is the sequence of states an Activity goes through from creation to destruction.

2. **Why is lifecycle important?**

To manage resources, UI state, and app performance correctly.

3. **Name the Activity lifecycle methods.**

onCreate(), onStart(), onResume(), onPause(), onStop(), onDestroy()

4. **Which method is called first?**

onCreate()

5. **Where do you set the layout?**

Inside onCreate() using setContentView().

6. **Which method makes the Activity visible?**

onStart()

7. **What is a Fragment?**

A Fragment is a reusable part of UI inside an Activity.

8. **Why use Fragments?**

For modular UI design and better handling of large screens.

9. **Name Fragment lifecycle methods.**

onAttach(), onCreate(), onCreateView(), onStart(), onResume(), onPause(), onStop(),

onDestroyView(), onDetach()

10. **Which method inflates Fragment layout?**

onCreateView()

Ex. No : 8

App Architecture – UI Layer

Aim

To understand the **UI layer architecture** in Android applications and implement user interaction by separating UI components and event-handling logic.

What is App Architecture?

- App architecture defines how different parts of an application are organized and interact with each other.
- A well-structured architecture makes the app:
    - Easy to understand
    - Easy to maintain
    - Easy to extend

UI Layer – Core Concept

- The UI Layer is responsible for:
    - Displaying data to the user
    - Handling user interactions (clicks, inputs)
- It consists of:
    - XML Layouts → UI design
    - Activity / Fragment → UI logic

Responsibilities of UI Layer

- Capture user actions (Button click, text input)
- Update UI elements dynamically
- Communicate with underlying logic (future ViewModel / data layer)

Procedure

1. Create a new Android project using Empty Activity.
2. Design the UI in activity_main.xml using TextView, EditText, and Button.
3. Open MainActivity.kt.
4. Handle user input and Button click inside the Activity.
5. Update TextView dynamically based on user action.
6. Run the app and observe UI behavior.

STEP 1: Create New Project

1. Open Android Studio
2. Click New Project
3. Select Empty Views Activity
4. Click Next
5. Enter:
    - Name: UILayerDemo
    - Language: Kotlin
    - Min SDK: API 21
6. Click Finish

STEP 2: Design UI (XML)

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp">
```

```
    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="UI Layer Demo"
        android:textSize="22sp"
        android:gravity="center"/>

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Message will appear here"
        android:textSize="18sp"
        android:paddingTop="20dp"/>

    <Button
        android:id="@+id/btnLoad"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Load Data"/>
</LinearLayout>
```

STEP 3: Create ViewModel

1. Right-click package name
2. Select New → Kotlin Class/File
3. Name: MainViewModel
4. Type: Class

```
package com.example.uilayerdemo

import androidx.lifecycle.ViewModel

class MainViewModel : ViewModel() {
    fun getMessage(): String {
        return "Hello from ViewModel (UI Layer)"
    }
}
```

STEP 4: MainActivity.kt

```
package com.example.uilayerdemo

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.activity.viewModels
import androidx.appcompat.app.AppCompatActivity


class MainActivity : AppCompatActivity() {
private val viewModel: MainViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
```

```
        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnLoad = findViewById<Button>(R.id.btnLoad)

        btnLoad.setOnClickListener {
            tvMessage.text = viewModel.getMessage()
        }
    }
}
```

STEP 5: Run Using Device Manager

1. Open Device Manager
2. Start Emulator
3. Click Run ▶


1:14
LTE
UI Layer Example
Welcome to MADL Lab
Enter your name
abc
Submit
Hello, abc 👋
abc
ABC

Viva - Questions

1. What is App Architecture in Android?

It is a structured way of organizing code to improve maintainability, testability, and scalability.

2. Why is architecture needed in Android apps?

To separate UI logic from business logic and avoid tightly coupled code.

3. What are the main layers in Android architecture?

- UI Layer
- Domain Layer (optional)
- Data Layer

4. What is the responsibility of the UI layer?

To display data, handle user interactions, and observe state changes.

5. What components belong to the UI layer?

- Activity
- Fragment
- ViewModel
- UI State (State holders)

6. Why should UI layer not contain business logic?

To keep UI simple and allow easy testing and reuse.

7. What is a ViewModel?

A class that stores and manages UI-related data in a lifecycle-aware way.

8. Why ViewModel is lifecycle-aware?

It survives configuration changes like screen rotation.

9. Where is ViewModel stored?

In memory and managed by the ViewModelStoreOwner.

10. Can ViewModel hold a Context?

+ **No (except Application context using AndroidViewModel).**

Ex. No :09

Aim

To understand the Persistence Layer in Android application architecture and implement data storage using SharedPreferences**.**

Explanation

What is App Architecture?

- App architecture organizes an application into layers so that each layer has a specific responsibility.
- Common layers include:
    - UI Layer
    - Business Logic Layer
    - Persistence (Data) Layer

Persistence Layer – Core Concept

- The Persistence Layer is responsible for:
    - Storing data permanently
    - Retrieving stored data when needed
- Data remains available even after:
    - App is closed
    - Device is restarted

Why Persistence is Important

- To save user information (name, settings)
- To maintain application state
- To improve user experience by avoiding repeated data entry

SharedPreferences

- SharedPreferences is a lightweight storage mechanism in Android.
- It stores data as key–value pairs.
- Suitable for storing small amounts of simple data such as:
    - Username
    - Login status
    - App preferences

Working of SharedPreferences

1. User enters data.
2. Data is stored using a unique key.
3. Data can be retrieved using the same key.
4. Stored data persists across app restarts**.**

Procedure

1. Create a new Android project using Empty Activity.
2. Design UI with EditText, Buttons, and TextView.
3. Use SharedPreferences to save user data.
4. Retrieve and display saved data.
5. Close and reopen the app to verify persistence.

Persistence Layer Components

1. Entity – Represents a table
2. DAO – Defines database operations
3. Database – Holds the database

Why Room Database?

- Official Android library
- Easy to use
- Avoids raw SQL errors
- Lifecycle aware

STEP 1: Create New Project

1. Open Android Studio
2. Click New Project
3. Choose Empty Views Activity
4. Click Next
5. Enter:
    - Name: PersistenceDemo
    - Language: Kotlin
    - Min SDK: API 21
6. Click Finish

STEP 2: Add Room Dependencies

Gradle Scripts → build.gradle (Module :app)

build.gradle (Module :app)

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.kapt)
}
```

STEP 4: Dependencies block (keep as-is)

```
dependencies {
    implementation(libs.androidx.room.runtime)
    implementation(libs.androidx.room.ktx)
    kapt(libs.androidx.room.compiler)
}
```

STEP 5: Sync + Clean (IMPORTANT ORDER)

1. **File → Sync Project with Gradle Files**

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="20dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Name"/>
```

```
    <Button
        android:id="@+id/btnSave"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Save to Database"/>

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Stored Data:"
        android:paddingTop="20dp"
        android:textSize="18sp"/>
</LinearLayout>
```

STEP 4: Create Entity Class

Right-click package → New → Kotlin Class/File

Name: UserEntity

```
package com.example.persistencedemo
import androidx.room.Entity
import  androidx.room.PrimaryKey
@Entity(tableName = "user_table")
data class UserEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val name: String
)
```

STEP 5: Create DAO Interface

New → Kotlin Class/File

Name: UserDao

Type: Interface

```
package com.example.persistencedemo


import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query


@Dao
interface UserDao {


    @Insert
    suspend fun insertUser(user: UserEntity)


    @Query("SELECT * FROM user_table")
    suspend fun getAllUsers(): List<UserEntity>
}
```

STEP 6: Create Database Class

```
package com.example.persistencedemo


import  android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase


@Database(entities = [UserEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {


    abstract fun userDao(): UserDao


    companion object {
        private var INSTANCE: AppDatabase? = null


        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "user_database"
                ).build().also {
                    INSTANCE = it
                }
            }
```

```
        }
    }
}
```

MainActivity.kt

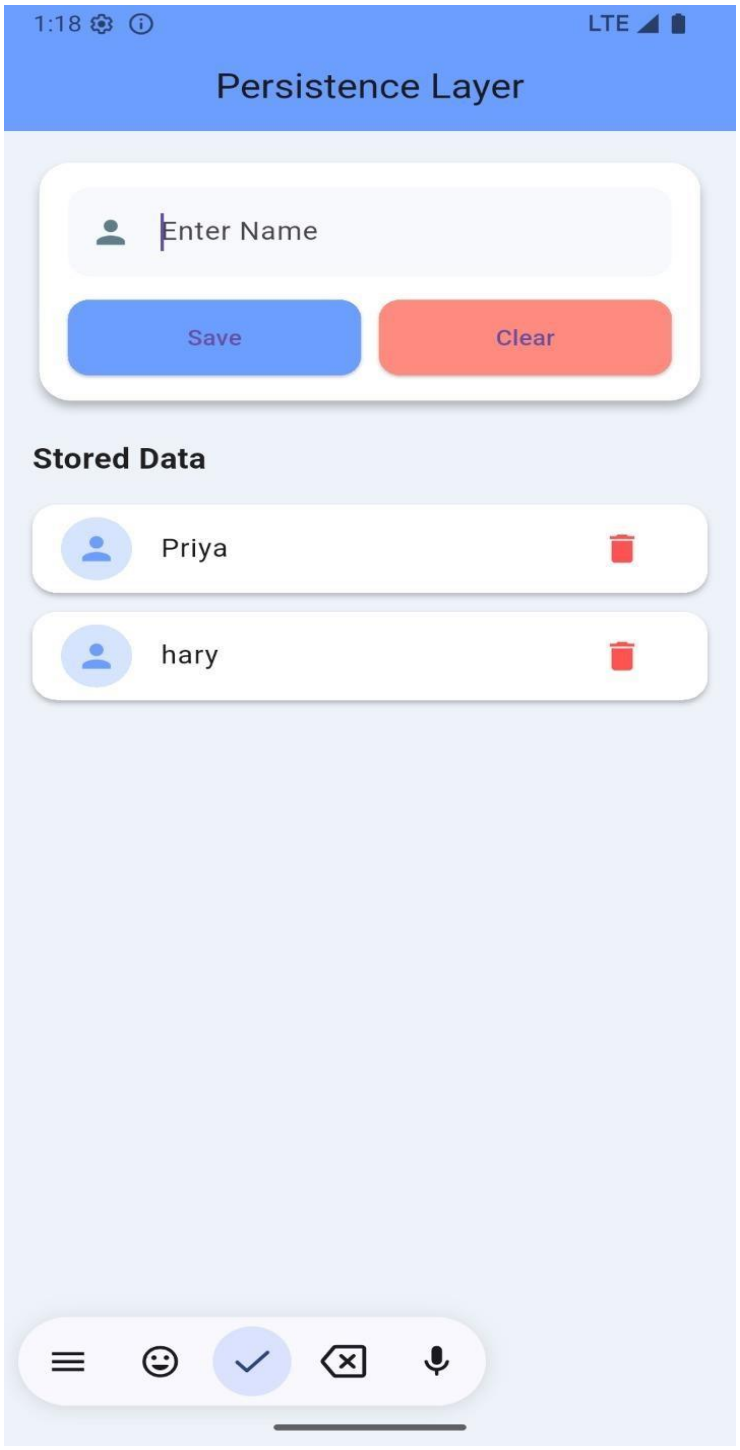
```
package com.example.persistencedemo

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etName = findViewById<EditText>(R.id.etName)
        val btnSave = findViewById<Button>(R.id.btnSave)
        val tvResult = findViewById<TextView>(R.id.tvResult)
        val db = AppDatabase.getDatabase(this)
```

```
        btnSave.setOnClickListener {
            val name = etName.text.toString()

            CoroutineScope(Dispatchers.IO).launch {
                db.userDao().insertUser(UserEntity(name = name))
                val users = db.userDao().getAllUsers()

                runOnUiThread {
                    tvResult.text = "Stored Data:\n"
                    users.forEach {
                        tvResult.append(it.name + "\n")
                    }
                }
            }
        }
    }
}
```

STEP 8: Run Using Device Manager

1. Open Device Manager
2. Start Emulator
3. Click Run ▶

OutPut


1:18
LTE
Persistence Layer
Enter Name
Save
Clear
Stored Data
Priya
hary

Viva Voce Questions

1. **What is Persistence Layer?**
2. **What is Room Database?**
3. **Purpose of Entity?**
4. **What is DAO?**
5. **Difference between UI Layer and Persistence Layer?**

Ex. No. : 10 Date

Advanced RecyclerView

Aim

To implement an Advanced RecyclerView in Android using Kotlin, which displays a list of items and handles item click events.

Software Requirements

- Android Studio Otter
- Language: Kotlin
- Minimum SDK: API 21
- Target SDK: 33 or 34

Step 1: Create a New Project (VERY IMPORTANT)

1. Open Android Studio Otter
2. Click New Project
3. Select Empty Views Activity
4. Click Next
5. Fill details:
    - Name: RecyclerViewAdvanced
    - Language: Kotlin
    - Minimum SDK: API 21
6. Click Finish
7. Wait until Gradle Build finishes completely ■

Step 2: Add RecyclerView Dependency

Open build.gradle (Module: app)

Add this inside dependencies { }

implementation "androidx.recyclerview:recyclerview:1.3.2"

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 4: Create Row Layout

res/layout/item_row.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/txtItem"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="20dp"
    android:textSize="18sp"
    android:textColor="#000000"/>
```

Step 5: Create Data Model

Right click package → New → Kotlin Class/File

Name: Item.kt

```
data class Item(val name: String)
```

Step 6: Create RecyclerView Adapter

ItemAdapter.kt

```
import android.view.LayoutInflater
import android.view.View
import  android.view.ViewGroup
import android.widget.TextView
import android.widget.Toast
import androidx.recyclerview.widget.RecyclerView

class ItemAdapter(private val itemList: List<Item>) :
    RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() {
```

```
class ItemViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
    val textView: TextView = itemView.findViewById(R.id.txtItem)
}

override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ItemViewHolder {
    val view = LayoutInflater.from(parent.context)
        .inflate(R.layout.item_row, parent, false)
    return ItemViewHolder(view)
}

override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
    holder.textView.text = itemList[position].name

    holder.itemView.setOnClickListener {
        Toast.makeText(
            holder.itemView.context,
            "Clicked: ${itemList[position].name}",
            Toast.LENGTH_SHORT
        ).show()
    }
}

override fun getItemCount(): Int {
    return itemList.size
}
}
```

MainActivity.kt

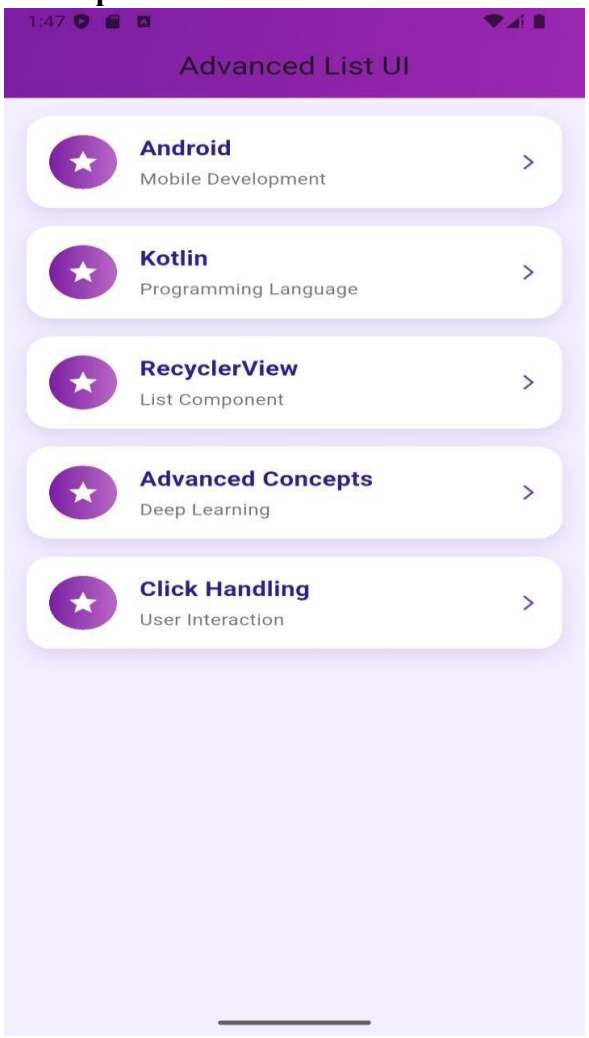
```
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val recyclerView = findViewById<RecyclerView>(R.id.recyclerView)
        recyclerView.layoutManager = LinearLayoutManager(this)

        val items = listOf(
            Item("Android"),
            Item("Kotlin"),
            Item("RecyclerView"),
            Item("Advanced Concepts"),
            Item("Click Handling")
        )

        val adapter = ItemAdapter(items)
        recyclerView.adapter = adapter
    }
```

}

Step 8: Run the App

1. **Select Emulator or Mobile Device**
2. **Click ▶ Run**
3. **Output:**
    - **List appears**
    - **Click any item → Toast message shown**

Output :



Viva -Questions

1. What is the aim of Experiment No. 10?

To implement an advanced RecyclerView in Android to display a list of items with click handling.

2. Which Android component is used to display large lists efficiently?

RecyclerView.

3. Which file contains the RecyclerView UI?

activity_main.xml

4. Which class connects data to RecyclerView?

Adapter class.

5. What data type is used to store RecyclerView items?

A data class or list (e.g., List<Item>).

6. Why is RecyclerView called "advanced" compared to ListView?

RecyclerView supports view recycling, flexible layouts, animations, and better performance.

7. What is the role of LayoutManager in this experiment?

It defines how list items are arranged (vertical list using LinearLayoutManager).

8. What is ViewHolder and why is it used?

ViewHolder holds item view references to reduce repeated view lookups and improve performance.

9. Explain the purpose of onCreateViewHolder().

It inflates the item layout and creates a ViewHolder object.

10. Explain the purpose of onBindViewHolder().

It binds data to the views at a specific position.

Experiment No. 11 – Connect to the Internet (Android – Kotlin)

Aim

To develop an Android application that connects to the Internet and loads a web page using WebView.

Software / Tools Required

- Android Studio Otter
- Language: Kotlin
- Minimum SDK: API 21
- Internet connection

- Android apps cannot access the Internet by default
- Permission must be declared in AndroidManifest.xml
- WebView is used to load and display web content inside an app

Procedure

Step 1: Create a New Project

1. Open Android Studio
2. New Project → Empty Views Activity
3. Name: InternetConnectApp
4. Language: Kotlin
5. Minimum SDK: API 21
6. Finish

Step 2: Add Internet Permission

AndroidManifest.xml

Add before <application> tag:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <WebView
        android:id="@+id/webView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>
```

MainActivity.kt

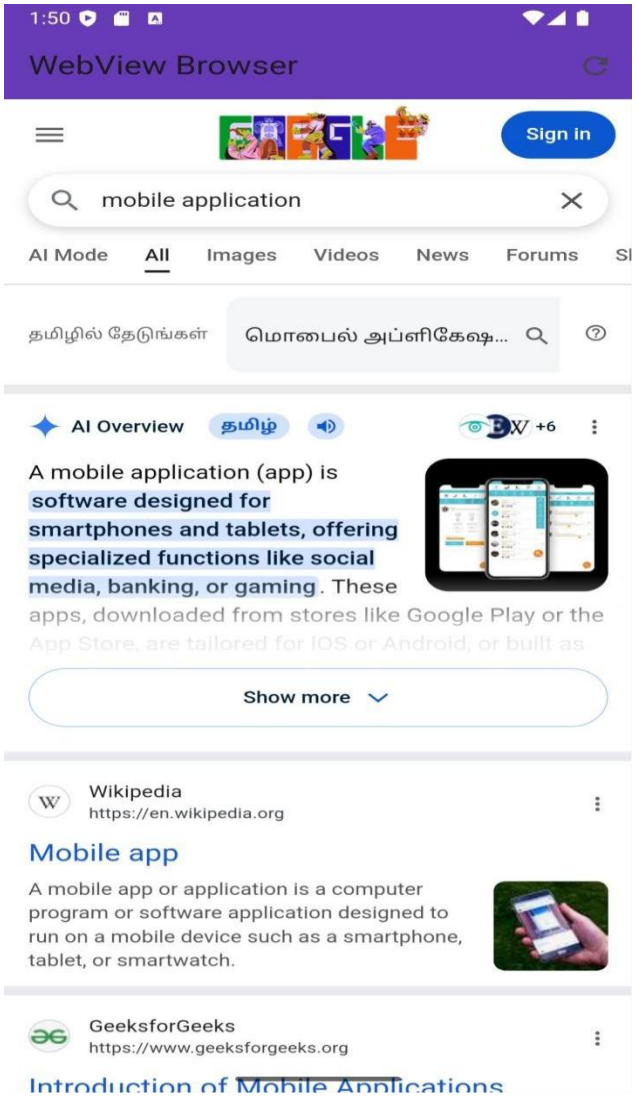
```
package com.example.internetconnectapp

import android.os.Bundle
import android.webkit.WebView
import android.webkit.WebViewClient
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val webView = findViewById<WebView>(R.id.webView)

        webView.webViewClient = WebViewClient()
        webView.settings.javaScriptEnabled = true
        webView.loadUrl("https://www.google.com")
    }
}
```

Step 5: Run the Application

1. Connect emulator or mobile device
2. Click Run ▶
3. Output:
    - Google page loads inside the app

Output :


1:50
WebView Browser
Sign in
mobile application
AI Mode
All
Images
Videos
News
Forums
தமிழில் தேடுங்கள்
மொபைல் அப்ளிகேஷ...
AI Overview
தமிழ்
+6
A mobile application (app) is software designed for smartphones and tablets, offering specialized functions like social media, banking, or gaming. These apps, downloaded from stores like Google Play or the App Store, are tailored for IOS or Android, or built as
Show more
Wikipedia
https://en.wikipedia.org
Mobile app
A mobile app or application is a computer program or software application designed to run on a mobile device such as a smartphone, tablet, or smartwatch.
GeeksforGeeks
https://www.geeksforgeeks.org
Introduction of Mobile Applications

Viva Voce – Experiment 11

1. What is the aim of this experiment?

To connect an Android application to the Internet.

2. Which permission is required to access the Internet?

android.permission.INTERNET

3. Where is Internet permission declared?

In AndroidManifest.xml

4. What is WebView?

A component used to display web pages inside an Android app.

5. Why is WebViewClient used?

To open links inside the app instead of the browser.

6. What happens if Internet permission is not given?

The app cannot load online content.

7. Can Android access Internet without permission?

No.

8. Which method loads a webpage?

loadUrl()

9. Is WebView part of AndroidX?

No, it is part of Android SDK.

10. Can WebView load local HTML files?

Yes.

Experiment No. 12

Implementation of Repository Pattern with Work Manager in Android

Aim

To implement the Repository Pattern for data handling and use WorkManager to perform a background task in an Android application.

Objectives

- To understand clean architecture using Repository Pattern
- To learn background task execution using WorkManager
- To integrate Repository with WorkManager

Repository Pattern

Repository Pattern is an architectural pattern that acts as a **single source of truth** for data.
It separates **data access logic** from UI logic and is commonly used in **MVVM architecture.**

WorkManager

WorkManager is a Jetpack library used to perform **reliable background tasks**, even if the app is closed or the device restarts.

Step 1: Create a New Project

1. Open Android Studio
2. New Project → Empty Views Activity
3. Name: RepoWorkManagerApp
4. Language: Kotlin
5. Minimum SDK: API 21
6. Finish

Step 2: Add WorkManager Dependency

build.gradle (Module: app)

implementation "androidx.work:work-runtime-ktx:2.9.0"

Step 3: Create Worker Class

`Right click package → New → Kotlin Class

Name: DataSyncWorker.kt

```
import android.content.Context
import androidx.work.Worker
import androidx.work.WorkerParameters

class DataSyncWorker(
    context: Context,
    params: WorkerParameters
) : Worker(context, params) {

    override fun doWork(): Result {
        println("Data syncing in background")
        return Result.success()
    }
}
```

Step 4: Create Repository Class

New → Kotlin Class

Name: DataRepository.kt

```
import android.content.Context
import androidx.work.OneTimeWorkRequestBuilder
import androidx.work.WorkManager

class DataRepository(private val context: Context) {
```

```
    fun syncData() {
        val workRequest =
            OneTimeWorkRequestBuilder<DataSyncWorker>().build()

        WorkManager.getInstance(context).enqueue(workRequest)
    }
}
```

Step 5: Update UI Layout

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <Button
        android:id="@+id/btnSync"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Sync Data"/>
</LinearLayout>
```

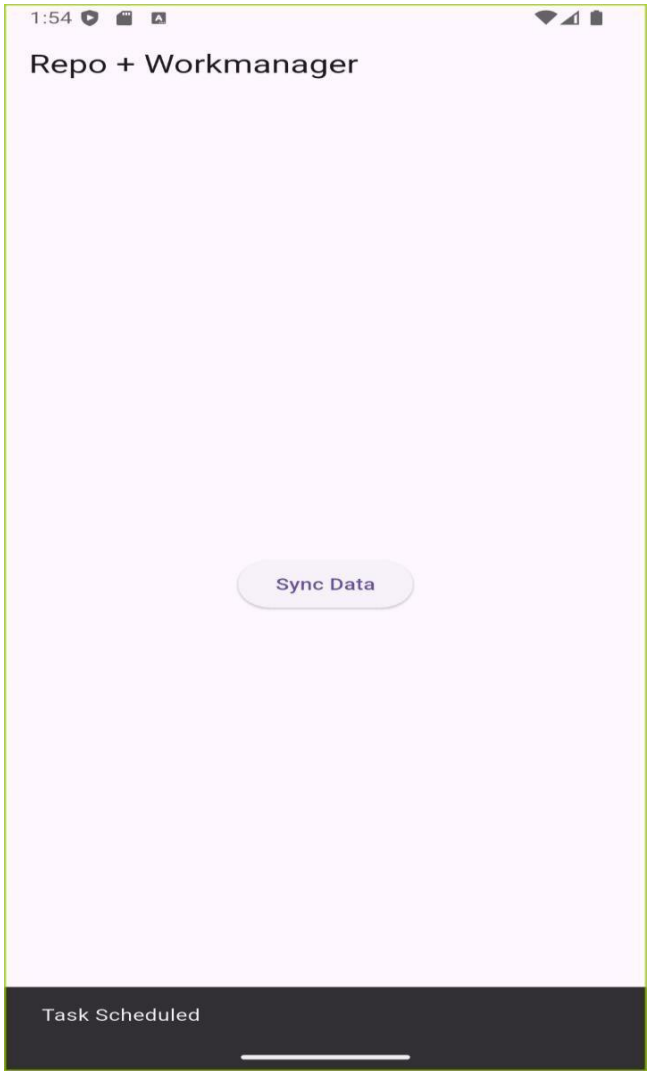
Step 6: MainActivity Code

MainActivity.kt

```
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val repository = DataRepository(this)

        val button = findViewById<Button>(R.id.btnSync)
        button.setOnClickListener {
            repository.syncData()
        }
    }
}
```

Output

- **When the Sync Data button is clicked**
- **Background task executes using WorkManager**
- **Data syncing message appears in Logcat**


1:54
Repo + Workmanager
Sync Data
Task Scheduled

Viva Voce

1. What is Repository Pattern?

It separates data access logic from UI.

2. Why Repository Pattern is used?

To improve code structure and maintainability.

3. What is WorkManager?

A Jetpack component used for background tasks.

4. When should WorkManager be used?

For guaranteed background execution.

5. What is a Worker class?

It defines the background task.

6. Can WorkManager run after app restart?

Yes.

7. How are Repository and WorkManager connected?

Repository schedules background tasks using WorkManager.

8. Which architecture supports Repository Pattern?

MVVM.

Experiment No. 13 – App UI Design (Android – Kotlin)

Designing a User Interface (UI) in Android using Layouts, Widgets, and Views

Aim

To design an interactive and user-friendly Android application UI using various layouts, widgets, and views.

Objectives

- Understand different layouts in Android (Linear, Relative, Constraint, Frame)
- Learn to use widgets like Button, TextView, EditText, ImageView
- Implement interactive UI components
- Make the UI visually appealing and responsive

- **Layout:** Defines how UI components are arranged
- **LinearLayout:** Arranges views vertically or horizontally
- **RelativeLayout:** Positions views relative to others
- **ConstraintLayout:** Flexible layout for modern UI design
- **Widgets / Views:** Interactive components like Button, TextView, EditText, ImageView
- **Styles / Themes:** Control color, font, and overall look of the app

Step 1: Create a New Project

1. Open Android Studio → New Project → Empty Activity
2. Name: AppUIDesign
3. Language: Kotlin
4. Minimum SDK: API 21
5. Finish

Step 2: Open Layout File

- Open res/layout/activity_main.xml
- Switch to Design View (drag-and-drop) or Code View (XML)

Step 3: Add Layout

Example **ConstraintLayout**:

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/txtTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome to My App"
        android:textSize="24sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <EditText
        android:id="@+id/edtName"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Enter your name"
```

```
        app:layout_constraintTop_toBottomOf="@id/txtTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_margin="16dp"/>

    <Button
        android:id="@+id/btnSubmit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Submit"
        app:layout_constraintTop_toBottomOf="@id/edtName"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="16dp"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

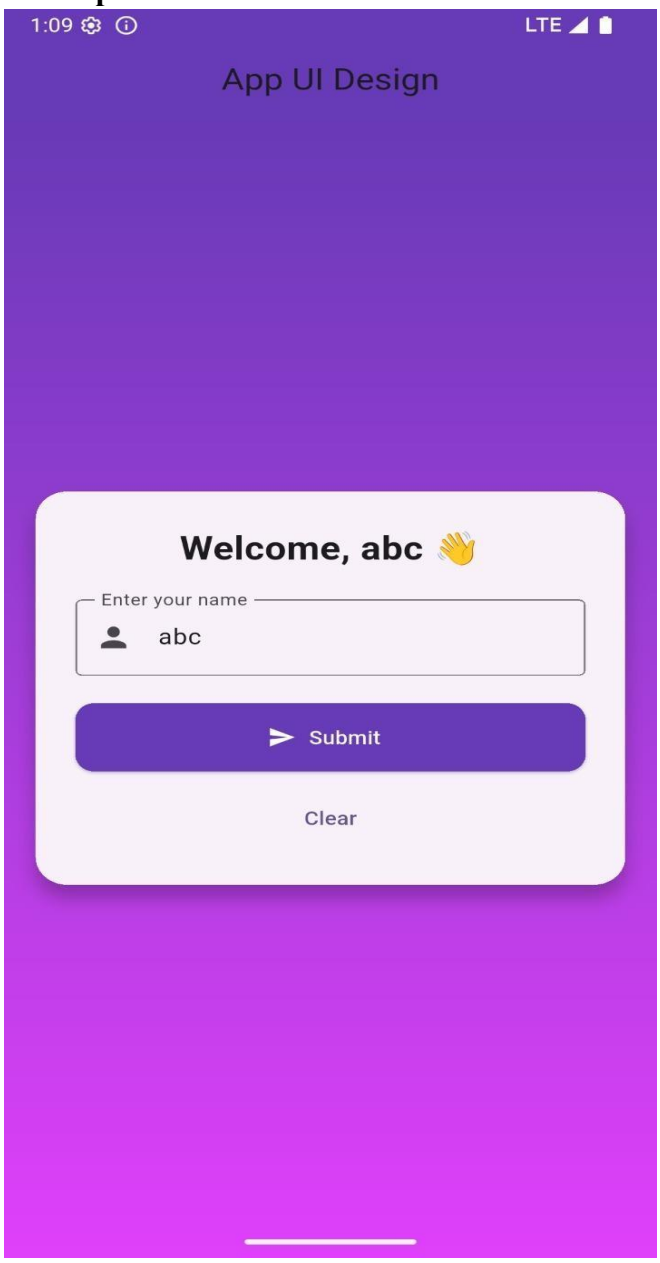
```
package com.example.appuidesign
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
```

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val edtName = findViewById<EditText>(R.id.edtName)
        val btnSubmit = findViewById<Button>(R.id.btnSubmit)
        val txtTitle = findViewById<TextView>(R.id.txtTitle)
        btnSubmit.setOnClickListener {
            val name = edtName.text.toString()
            if (name.isNotEmpty()) {
                Toast.makeText(this, "Hello, $name!", Toast.LENGTH_SHORT).show()
                txtTitle.text = "Welcome, $name"
            } else {
                Toast.makeText(this, "Please enter your name", Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

Step 5: Run the Application

1. Connect Emulator / Device
2. Click Run ▶
3. Output:
    - TextView displays welcome text
    - EditText accepts input
    - Button click shows Toast and updates TextView

Output :


1:09
LTE
App UI Design
Welcome, abc 👋
Enter your name
abc
Submit
Clear

1. What is a Layout?

Defines how UI components are arranged on the screen.

2. Name some layouts in Android.

LinearLayout, RelativeLayout, ConstraintLayout, FrameLayout.

3. What is a Widget?

A UI component that allows interaction, e.g., Button, EditText, TextView.

4. What is ConstraintLayout?

A flexible layout that lets you define constraints for positioning views relative to others or parent.

5. How do you handle a button click?

Using setOnClickListener in Kotlin.

6. What is a Toast?

A small popup message displayed for a short duration.

7. How do you make UI responsive for different screens?

By using dp units, ConstraintLayout, and wrap_content / match_parent.

8. Where are the resources like strings and colors stored?

In res/values/strings.xml and res/values/colors.xml.